

# Vue中computed计算属性的核心概念、用法、对比及注意事项

## 一、computed 计算属性核心概念

`computed` 是 Vue (Vue2/Vue3) 中用于**声明式推导/计算数据**的核心特性，依赖响应式数据缓存，只有依赖项变化时才重新计算，本质是“带缓存的响应式计算函数”。

核心特点：

1. **缓存性**：依赖项不变时，多次访问计算属性会直接返回缓存结果，不重复执行计算逻辑；
2. **响应式**：依赖项更新时，自动重新计算并触发视图更新；
3. **声明式**：写法更接近“数据”而非“函数”，符合 Vue 数据驱动的设计思想。

## 二、基本用法（Vue2 vs Vue3）

### 1. Vue2 选项式 API 用法

Code block

```
1  new Vue({
2    el: '#app',
3    data() {
4      return {
5        firstName: 'Zhang',
6        lastName: 'San',
7        scores: [80, 90, 95]
8      };
9    },
10   computed: {
11     // 1. 只读计算属性（最常用）
12     fullName() {
13       return `${this.firstName} ${this.lastName}`;
14     },
15
16     // 2. 可读写计算属性（get/set）
17     fullNameEditable: {
18       get() {
19         return `${this.firstName} ${this.lastName}`;
20       },
```

```

21     set(newValue) {
22         const [first, last] = newValue.split(' ');
23         this.firstName = first || '';
24         this.lastName = last || '';
25     }
26 },
27
28 // 3. 依赖多个数据的计算
29 averageScore() {
30     return this.scores.reduce((sum, score) => sum + score, 0) /
    this.scores.length;
31 }
32 }
33 });

```

模板中使用：

Code block

```

1 <div>{{ fullName }}</div>
2 <div>{{ averageScore }}</div>
3 <!-- 可写计算属性赋值 -->
4 <input v-model="fullNameEditable" />

```

## 2. Vue3 组合式 API 用法

Vue3 中通过 `computed` 函数创建，返回一个只读的响应式引用对象（需 `.value` 访问）。

Code block

```

1 import { ref, computed } from 'vue';
2
3 export default {
4     setup() {
5         const firstName = ref('Zhang');
6         const lastName = ref('San');
7         const scores = ref([80, 90, 95]);
8
9         // 1. 只读计算属性
10        const fullName = computed(() => {
11            return `${firstName.value} ${lastName.value}`;
12        });
13
14        // 2. 可读写计算属性
15        const fullNameEditable = computed({
16            get() {

```

```
17         return `${firstName.value} ${lastName.value}`;
18     },
19     set(newValue) {
20         const [first, last] = newValue.split(' ');
21         firstName.value = first || '';
22         lastName.value = last || '';
23     }
24 });
25
26 // 3. 依赖多个响应式数据
27 const averageScore = computed(() => {
28     return scores.value.reduce((sum, score) => sum + score, 0) /
29     scores.value.length;
30 });
31
32 return { fullName, fullNameEditable, averageScore };
33 };
```

模板中使用（无需 `.value`，Vue 自动解包）：

Code block

```
1 <div>{{ fullName }}</div>
2 <input v-model="fullNameEditable" />
```

### 三、computed 与其他方案的对比

#### 1. computed vs methods（方法）

| 特性    | computed 计算属性    | methods 方法       |
|-------|------------------|------------------|
| 缓存性   | 依赖项不变时，多次访问返回缓存  | 每次调用都会重新执行函数     |
| 调用方式  | 模板中作为“属性”使用（无括号） | 模板中作为“函数”调用（需括号） |
| 响应式触发 | 依赖项更新时自动重新计算     | 每次触发重新渲染都会执行     |
| 适用场景  | 基于现有数据的推导（无副作用）  | 处理业务逻辑、有副作用的操作   |

示例对比：

Code block

```
1  // Vue2 示例
2  data() {
3    return { a: 1, b: 2 };
4  },
5  computed: {
6    sum() {
7      console.log('computed 执行');
8      return this.a + this.b;
9    }
10 },
11 methods: {
12   getSum() {
13     console.log('methods 执行');
14     return this.a + this.b;
15   }
16 }
```

模板中使用：

Code block

```
1  <!-- 多次访问 sum，仅首次打印「computed 执行」，依赖项不变时复用缓存 -->
2  <div>{{ sum }}</div>
3  <div>{{ sum }}</div>
4
5  <!-- 多次调用 getSum，每次都打印「methods 执行」 -->
6  <div>{{ getSum() }}</div>
7  <div>{{ getSum() }}</div>
```

2. computed vs watch（侦听器）

| 特性   | computed 计算属性   | watch 侦听器        |
|------|-----------------|------------------|
| 核心逻辑 | 声明式推导数据（从数据到数据） | 命令式响应数据变化（执行副作用） |
| 返回值  | 有返回值（可直接在模板使用）  | 无返回值（仅执行回调）      |
| 触发时机 | 依赖项更新时自动计算      | 监听的数据源更新时执行回调    |
|      |                 |                  |

|      |             |                  |
|------|-------------|------------------|
| 适用场景 | 数据推导、格式化、过滤 | 异步操作、复杂业务逻辑（如请求） |
|------|-------------|------------------|

示例对比：

Code block

```
1  // 场景：根据输入的价格（元）计算美元价格（汇率 7.2）
2  data() {
3      return { priceCny: 100, exchangeRate: 7.2 };
4  },
5
6  // computed 实现（推荐，声明式）
7  computed: {
8      priceUsd() {
9          return (this.priceCny / this.exchangeRate).toFixed(2);
10     }
11 },
12
13 // watch 实现（冗余，仅演示）
14 watch: {
15     priceCny: {
16         handler(newVal) {
17             this.priceUsd = (newVal / this.exchangeRate).toFixed(2);
18         },
19         immediate: true
20     }
21 },
22 data() {
23     return { priceCny: 100, exchangeRate: 7.2, priceUsd: '' };
24 }
```

3. computed vs ref/reactive（原始响应式数据）

|      |                 |                   |
|------|-----------------|-------------------|
| 特性   | computed 计算属性   | ref/reactive      |
| 本质   | 派生响应式数据（依赖其他数据） | 原始响应式数据（独立数据源）    |
| 可修改性 | 只读（除非配置 set）    | 可直接修改（.value 或赋值） |
| 缓存   | 有缓存             | 无缓存（赋值即更新）        |
| 适用场景 | 数据推导            | 存储原始业务数据          |

## 四、computed 高级用法与注意事项

### 1. 避免副作用

computed 应仅做**纯数据推导**，禁止包含异步操作、DOM 修改、状态修改等副作用（这些逻辑应放在 methods/watch 中）。

Code block

```
1 // 错误示例: computed 中包含异步副作用
2 computed: {
3   asyncData() {
4     // 禁止! computed 不能是异步函数
5     fetch('/api/data').then(res => res.json()).then(data => {
6       return data;
7     });
8   }
9 }
```

### 2. 依赖项的注意事项

- computed 仅追踪**响应式依赖**（data/ref/reactive 声明的属性），非响应式数据不会触发重新计算；
- 避免在 computed 中使用 Date.now()、Math.random() 等非响应式值，否则计算结果不会更新。

### 3. Vue3 中计算属性的解包

- 模板中自动解包：{{ fullName }} 无需 .value ；
- 脚本中需手动解包：console.log(fullName.value) ；
- 嵌套在 reactive 对象中自动解包：

Code block

```
1 const obj = reactive({ fullName });
2 console.log(obj.fullName); // 无需 .value
```

## 五、总结

| 方案 | 核心优势 | 适用场景 |
|----|------|------|
|    |      |      |

|          |               |                   |
|----------|---------------|-------------------|
| computed | 缓存、声明式、简洁     | 数据推导、格式化、过滤（无副作用） |
| methods  | 灵活、支持副作用      | 业务逻辑、事件处理、按需执行    |
| watch    | 响应变化、支持异步/副作用 | 数据变化后的复杂操作（如请求）   |

核心建议：

- 只要是“基于现有响应式数据推导新数据”，优先用 `computed`；
- 需要“执行操作/副作用”，用 `methods` 或 `watch`；
- 避免滥用 `watch` 实现数据推导，也避免用 `methods` 替代 `computed` 导致性能浪费。

（注：文档部分内容可能由 AI 生成）